TECHNICAL REFERENCE GUIDE

Qwik Guide: MLflow

Tracking · Models · Registry · Projects · MLOps

> MLflow is an open-source platform for managing the machine learning lifecycle. Created by Databricks, it gives teams a consistent way to *track experiments, package and version models, and deploy them* — replacing the ad-hoc mix of scripts, spreadsheets, and naming conventions that most projects accumulate. It is framework-agnostic and sits squarely in the deployment-and-monitoring half of the ML lifecycle.

1 Why MLflow Exists

As a project grows, a familiar problem appears: dozens of training runs, each with slightly different hyperparameters, data versions, and results — and no reliable record of which produced what. “Which run gave us that 94% model?” becomes an unanswerable question.

MLflow addresses this with four core components that work together but can also be adopted independently:

① **Tracking** — Log parameters, metrics, code versions, and output artifacts for every run so you can compare and reproduce them.

② **Models** — A standard packaging format that lets you save a trained model once and serve it in many environments.

③ **Model Registry** — A central store for versioning models, managing stage transitions, and adding approvals.

④ **Projects** — A convention for packaging code with its dependencies so runs are reproducible across machines.

> 💡 **Tip:** You don’t have to adopt all four at once. Most teams start with Tracking — it delivers immediate value with a single line of code — and grow into the Registry as models head to

production.

2 Tracking: The Experiment Log

Tracking is the heart of MLflow. Every training **run** can record four kinds of data: **parameters** (inputs you set, like learning rate), **metrics** (numeric results, like accuracy or loss), **artifacts** (output files — the model, plots, data samples), and **metadata** (code version, start time, source).

Instrumenting a run is deliberately lightweight:

```
import mlflow

with mlflow.start_run():
    mlflow.log_param("learning_rate", 0.01)
    mlflow.log_param("batch_size", 32)

    # ... train the model ...

    mlflow.log_metric("accuracy", 0.94)
    mlflow.log_metric("loss", 0.21)
    mlflow.log_artifact("confusion_matrix.png")
    mlflow.sklearn.log_model(model, "model")
```

Runs are grouped into **experiments**, and the MLflow UI lets you sort, filter, and compare them side by side — plotting accuracy against learning rate across a hundred runs, for instance, to see which configuration wins.

> 💡 **Tip:** Use `mlflow.autolog()` to capture parameters, metrics, and the model automatically for many frameworks (scikit-learn, PyTorch, XGBoost, and more) — often without changing your training code at all.

3 Models: Package Once, Serve Anywhere

A model trained in scikit-learn and one trained in PyTorch normally require completely different serving code. The MLflow **Model** format solves this by saving a model together with its dependencies and one or more “flavors” — standardized interfaces that downstream tools know how to load.

The saved model directory contains everything needed to reproduce and serve it:

```
my_model/
  MLmodel            # metadata: flavors, signature, version
  model.pkl          # the serialized model
  conda.yaml         # environment specification
  requirements.txt   # pip dependencies
  input_example.json # sample input for validation
```

Because the format is standardized, the same packaged model can be deployed to a REST endpoint, run as a batch scoring job, loaded into Spark, or pushed to a cloud platform — without rewriting it for each target. Spinning up a local inference server is a single command:

```
mlflow models serve -m runs:/<run_id>/model -p 5000
```

> 💡 **Tip:** Log a **model signature** (the expected input and output schema). It lets MLflow validate incoming requests at serving time and catch schema mismatches before they become silent production errors.

4 The Model Registry: Governance & Promotion

Tracking answers “what did we try?” The **Registry** answers “what is running in production, and how did it get there?” It is a central hub where registered models carry versions, stage assignments, and an audit trail of who changed what.

Each registered model moves through a controlled lifecycle of stages:

```
None → Staging → Production → Archived
```

Versioning

Registering a new model under an existing name auto-increments its version. You always know exactly which artifact each version points to, and can roll back instantly.

Stage Transitions & Approvals

Promoting a model from Staging to Production can require review and sign-off, with annotations recording the rationale — the governance layer the deployment patterns in the ML Overview guide depend on.

Aliases & Loading by Stage

Serving code can reference `models:/fraud_detector@champion` instead of a hard-coded version. Promote a new version to that alias and traffic shifts with no code change.

> ⚠ **Warning:** The classic stage names (Staging, Production) are being superseded by **aliases** and **tags** in newer MLflow versions. Check your version's docs before standardizing a promotion workflow — the concepts carry over, but the API differs.

5 Projects & Reproducibility

An **MLflow Project** is simply a directory with an `MLproject` file that declares the environment and named entry points. It turns "it works on my machine" into a command anyone can run identically:

```
name: churn_model

python_env: python_env.yaml

entry_points:
  main:
    parameters:
      learning_rate: {type: float, default: 0.01}
    command: "python train.py --lr {learning_rate}"
```

Anyone with the project — local, a teammate, or CI — runs it the same way, and MLflow recreates the declared environment first:

```
mlflow run . -P learning_rate=0.05
```

6 Deployment Architecture

How you run MLflow scales with your team. The same API works from a laptop to a shared platform — only the backing stores change.

Local mode — Runs log to an `mlruns/` folder on disk. Zero setup; perfect for solo experimentation.

Tracking Server — A shared server all team members log to, so experiments live in one place instead of scattered laptops.

Backend store — A database (e.g., PostgreSQL) holds run metadata, parameters, and metrics — the structured, queryable data.

Artifact store — Blob storage (S3, GCS, Azure Blob) holds the large files: models, plots, and datasets.

The split matters: metadata is small and queryable, so it belongs in a database; artifacts are large binaries, so they belong in object storage. Separating them is what lets a tracking server stay responsive as run history grows into the millions.

7 MLflow for GenAI & LLMs

Recent MLflow releases extend well beyond classical ML into generative AI, reflecting where much of the field's attention has moved:

Tracing & Observability — Capture the full execution of an LLM or agent app — prompts, tool calls, retrieved context, and latency at each step — for debugging and monitoring.

Prompt Management — Version and track prompts as first-class artifacts, so a prompt change is as auditable as a code or model change.

LLM Evaluation — Built-in utilities to score model outputs against metrics — including LLM-as-judge approaches — and compare candidates systematically rather than by eyeballing samples.

> 💡 **Tip:** The mental model carries over cleanly: a prompt is a parameter, an eval score is a metric, and a trace is an artifact. If you already track classical experiments, tracking GenAI ones will feel familiar.

8 Where MLflow Fits in the Lifecycle

Mapping the components back to the four-stage ML lifecycle shows MLflow's reach — and its edges:

```
Scoping    →  (not MLflow's job)
Data       →  log dataset versions as artifacts
Modeling   →  Tracking + Projects   ★ core strength
Deployment →  Models + Registry     ★ core strength
```

MLflow is strongest across **modeling and deployment**. It is not a feature store, a data pipeline orchestrator, or a monitoring dashboard on its own — it complements tools like TFX, Feast, and Airflow rather than replacing them.

> ⚠ **Warning:** MLflow tracks and packages models — it does not, by itself, watch for data drift or concept drift in production. Pair it with dedicated monitoring (see the ML Overview guide) to close the loop.

★ Further Resources

Documentation & Getting Started

- MLflow Official Documentation
- MLflow on GitHub — source, issues, and examples
- Getting Started Tutorials

Component Deep Dives

- Tracking — experiments, runs, and the UI
- Model Registry — versioning and stage management
- Models — flavors, signatures, and serving

Related in This Series

- ml-ops.org — MLOps principles and community
- Managed MLflow on Databricks